

Spec: Model-Specific Inference for Qwen3.8-27B on DGX Spark

Oct 8, 2026 · @Someone

Summary

Decoding Qwen3.8-27B on a DGX Spark is bound by its 273 GB/s memory bus, not by compute. Every generated token streams about 15 GB of 4-bit weights through that bus, which caps plain decoding near 17 tokens/s however good the kernels are. The best public recipe today, SGLang with the DFlash2 drafter, reaches 47.9 tokens/s and a 232 ms first token on this machine. The plan is Husky's: get more tokens out of each weight read, then remove every other cost.

The target is Qwen3.8-27B, the Apache-2.0 dense member of the Qwen3.8 family released in August 2026: 64 layers in a 3:1 mix of Gated DeltaNet (linear attention) and gated full attention, a 248k vocabulary, and a trained DFlash2 block drafter already available. The other two family members do not fit the job: Qwen3.8-Max (2.4T, 95B active) needs four Sparks, and Qwen3.8-Flash-Next (176B MoE) runs at 28.5 tokens/s on one Spark with part of its weights on NVMe. We build a single-model engine for Qwen3.8-27B on the GB10 chip in two parts: an offline compiler that quantizes to NVFP4, folds the norms, lays the bytes out on disk in kernel order, instantiates one fused CUDA kernel per matrix shape and records each decode step as a CUDA graph; and a resident runtime that keeps each conversation's KV cache and DeltaNet state on the GPU.

Four levers, in order of expected payoff:

1. Speculative decoding tuned for this bus: the DFlash2 drafter quantized to NVFP4 with a reduced-vocabulary head, verification blocks of 8–32 rows sized for Spark's compute surplus, and prompt-lookup drafts spliced in for edit tasks.
2. Shape-specialized fused CUDA kernels over pre-tiled NVFP4 weights (norm + QKVZ projection + conv + delta rule; norm + gate·up; down + residual), so a 16-row verify costs little more than a 1-row step.
3. FP8 KV cache and a replay scheme for the DeltaNet recurrent state, so speculation on the hybrid architecture costs one state read and write per step instead of eight.
4. One CUDA graph per step with GPU-side sampling and resident sessions, taking host overhead to under 1 ms and a follow-up's first token to one weight pass.

Headline targets (estimates until M1 measures the real bus): 60 tokens/s or more on thinking traces and prose, 90 or more on code and math, 150 or more on edit tasks, first token of a follow-up in 80 ms or less, and greedy output identical to non-speculative decoding at the same precision. Against the 47.9 tokens/s recipe that is 1.3× on prose, 2× on code and 3× on edits; against plain decoding it is 3.5× to 9×. Husky's 4.5× does not transfer whole: Woof reads 2.4 GB per token on a faster bus, Qwen3.8-27B reads 15 GB on a slower one, so every lever here multiplies a smaller base.

Goals and non-goals

The engine runs exactly one model, Qwen3.8-27B, and every design choice may assume that.

Goals:

- Decode at or above 90% of the memory-bandwidth bound for a single sequence, then multiply that by speculation.
- Produce the same output as plain decoding: identical tokens under greedy sampling, the same distribution under temperature sampling, via exact speculative acceptance.
- Answer a follow-up message in a cached conversation with first-token latency of one weight pass plus overhead, 80 ms or less.
- Hold at least ten 32k-token conversations resident on the GPU, given 128 GB of unified memory and a 32 KB-per-token FP8 KV cache.
- Serve an OpenAI-compatible streaming API so existing clients work unchanged, including `reasoning_effort`, `enable_thinking` and `preserve_thinking`.
- Support thinking and non-thinking modes, the Qwen3 tool-call format, and image input through the model's own vision encoder.

Non-goals:

- Running any other model, including Qwen3.8-Flash-Next or Qwen3.6-27B. A new checkpoint with different shapes means recompiling.
- Multi-node inference across two Sparks over ConnectX-7.
- Training or fine-tuning the target model. Retraining the DFlash2 drafter for a bigger block is in scope.
- New quantization research. We start from NVIDIA's NVFP4 mixed-precision recipe and only decide which tensors stay at FP8.
- Video input and contexts beyond the native 262k tokens, in the first release.

- High-concurrency serving. The design centres on one user or one agent; a small-batch mode of 2–4 sequences is a stretch goal.

Hardware: what the GB10 gives and takes away

The DGX Spark is a compute-rich, bandwidth-poor machine, which is exactly the profile speculative decoding exploits. Its GB10 pairs a Blackwell GPU (compute capability 12.1, 5th-generation tensor cores with native FP4 and FP8) with a 20-core Arm CPU on one coherent 128 GB pool of LPDDR5x behind a 256-bit, 273 GB/s bus ([NVIDIA DGX Spark hardware docs](#)). NVIDIA rates it at 1 PFLOP of FP4 with sparsity, about 500 TFLOPS dense FP4 and 250 TFLOPS dense FP8, with a 24 MB GPU L2 (approximate, from NVIDIA's Hot Chips 2025 talk). The GB10 TDP is 140 W. Two copy engines, a 4 TB NVMe and a 200 Gb/s ConnectX-7 round it out; the ConnectX-7 is out of scope here.

Quantity	Value	Consequence for this design
Memory bandwidth	273 GB/s, shared by CPU and GPU	A 15 GB weight pass takes 55 ms at peak; nothing can beat ~18 tokens/s per weight read
Dense FP8 compute	~250 TFLOPS (derived from 1 PFLOP sparse FP4)	~900 FLOP per byte read: verifying 32 rows per pass costs compute but no bandwidth
Unified memory	128 GB (121 GB usable per sparkrun)	Weights (16 GB) plus 100 GB of sessions resident; there is no separate host RAM to offload to
GPU L2	~24 MB	The 3.1 MB-per-layer DeltaNet state and all activations stay on chip between kernels
Observed generic engines	38 tokens/s for Llama 3.1 8B q4_K_M in Ollama, i.e. ~190 GB/s effective	Generic runtimes reach 65–75% of the bus; our target is 90%

The arithmetic that shapes everything else: a single-row decode step does about 2 FLOP per weight, so at 4.5 bits per weight it runs the tensor cores at well under 1% of peak. Adding rows to the same pass is almost free until roughly 200 rows. In practice the limit

comes sooner, because tensor-core tiles below 64 rows run inefficiently and activations start to cost, so this design treats 32 rows as the practical verify ceiling and 8–16 as the default.

The bus must be measured, not assumed. Peak is 273 GB/s; sustained GPU streaming on LPDDR5x usually lands lower, and the CPU's own traffic shares the same pins. M0 measures a weight-streaming microbenchmark on the actual box, and every target in this document is restated against that number. The plan below assumes 90% of peak, 245 GB/s, is reachable by a well-written kernel.

One Spark-specific simplification: on this machine "GPU memory" and "host memory" are the same DRAM. There is nothing to offload sessions to, so the engine manages one large pool with its own paging and eviction, and never copies state across a PCIe bus that does not exist.

Sources: [DGX Spark hardware overview](#) · [GB10 at Hot Chips 2025](#) · [Ollama DGX Spark performance](#) · [llama.cpp on DGX Spark](#)

Model: Qwen3.8-27B shapes and the per-step byte budget

Qwen3.8-27B is a dense hybrid: 64 layers laid out as 16 repeats of three Gated DeltaNet blocks followed by one gated full-attention block, each with its own SwiGLU FFN, hidden size 5120, a 248,320-token padded vocabulary with untied embedding and output head, a native 262,144-token context, a vision encoder, and a trained MTP module ([model card](#)). Only the 16 attention layers keep a KV cache; the 48 DeltaNet layers keep a fixed-size recurrent state instead. Every matrix below has one size, which is what the compiler specializes on.

Component	Count	Shape (in × out)	Params	NVFP4 bytes per pass
DeltaNet in-projection (q, k, v, z fused)	48	5120 × 16,384	83.9M	47.2 MB
DeltaNet β/α projection, conv1d (k=4), gated norm	48	5120 × 96; 10,240 × 4	0.5M	~1 MB (kept BF16)
DeltaNet out-projection	48	6144 × 5120	31.5M	17.7 MB
Attention q-projection (query + output gate)	16	5120 × 12,288	62.9M	35.4 MB

Component	Count	Shape (in × out)	Params	NVFP4 bytes per pass
Attention k- and v-projections	16	5120 × 1024 each	10.5M	5.9 MB
Attention out-projection	16	6144 × 5120	31.5M	17.7 MB
FFN gate + up (fused)	64	5120 × 34,816	178.3M	100.3 MB
FFN down	64	17,408 × 5120	89.1M	50.1 MB
Transformer stack total	64		24.35B	13.70 GB
LM head	1	5120 × 248,320	1.27B	0.72 GB (FP8: 1.27 GB; BF16: 2.54 GB)
Token embedding	1	248,320 × 5120	1.27B	10 KB per token (row lookup)
Vision encoder	1		~1B	0 in the text decode path

NVFP4 is counted at 4.5 bits per weight: 4-bit values with one FP8 scale per 16 values. Figures are derived from the shapes on the model card; the published NVFP4 checkpoints are mixed-precision and come out larger, so M1 confirms the real byte count tensor by tensor.

State per sequence is small. The FP8 KV cache costs 16 layers × 2 × 4 heads × 256 dims = 32 KB per token: 1.05 GB at 32k tokens, 8.6 GB at the native 262k. The DeltaNet recurrent state is 48 layers × 48 heads × 128 × 128 values, 151 MB in FP32, independent of context length. The conv1d window adds 3 MB.

The per-step byte budget follows: 13.70 GB of layer weights, 0.72 GB of LM head, 0.30 GB of DeltaNet state read and written, plus the KV read at the current context: 14.9 GB at 4k tokens, 15.8 GB at 32k, 23.4 GB at 262k. At 273 GB/s those are 55, 58 and 86 ms per pass: 18, 17 and 12 tokens/s for one token per pass. For comparison, the public [RadixArk NVFP4 checkpoint with a BF16 head](#) loads 22.1 GB and reads about 17.6 GB per pass, a 15.5 tokens/s ceiling.

Details the kernels must honour:

- Gated attention: 24 query heads and 4 KV heads of dimension 256; the q-projection emits a query and an output gate per head; q and k get a per-head RMSNorm before RoPE; RoPE is partial, 64 of 256 dimensions, in the interleaved M-RoPE form with sections [11, 11, 10] shared with the vision tokens.
- Gated DeltaNet: 16 key/query heads and 48 value heads of dimension 128, a causal conv1d of width 4 with SiLU on q, k and v, per-head decay α and write strength β , and a gated RMSNorm on the output multiplied by SiLU(z).
- Thinking is on by default with `reasoning_effort` xhigh, medium or low, and `preserve_thinking` keeps earlier thinking blocks in the history by default, so the token prefix of a conversation is stable across turns. Recommended sampling: temperature 1.0, top-p 0.95, top-k 20 in thinking mode; temperature 0.7, top-p 0.8, top-k 20, presence penalty 1.5 in instruct mode.
- The DFlash2 drafter ([z-lab/Qwen3.8-27B-DFlash2](#)) is a 5-layer sliding-window block-diffusion model of about 1.9B parameters (3.58 GiB in BF16) with a top-K candidate selector; it conditions on the target's hidden states at fixed layers, drafts a block of 8 (7 draft tokens plus the bonus token) in one pass, and has no output head of its own, so it reuses the target's LM head ([W8 card](#)).

Sources: [Qwen/Qwen3.8-27B model card](#) · [Qwen3.8-27B on DGX Spark](#), [OpenZeka · DFlash2 drafter W8 card](#)

Performance targets

Every target is tokens accepted per step divided by step time, so the table separates the two. Baselines are the physical ceiling for one token per pass and the measured public recipe (SGLang, RadixArk NVFP4 with BF16 head, DFlash2 block 8, FP8 KV, 128-token prompts) on a single Spark. Targets assume 245 GB/s achieved and the acceptance lengths the DFlash2 community reports, ~6.0 tokens per step on math and ~3.3 on long prose; they are estimates until M1 and M2 replace them with measurements.

Metric	Physical ceiling, 1 token per pass	Public recipe today	Target	Stretch
Decode, thinking traces and prose (tokens/s)	18	47.9 (mixed 128-token prompts)	≥ 60	75

Metric	Physical ceiling, 1 token per pass	Public recipe today	Target	Stretch
Decode, code and math (tokens/s)	18	not published separately	≥ 90	110
Decode, edit tasks: typo fix, rename, JSON field (tokens/s)	18	not published	≥ 150	220
Step time at 4k context (ms)	55	~90 (inferred from 47.9 tokens/s at ~4.3 accepted)	≤ 72	65
First token, follow-up in a cached chat (ms)	55	232 (fresh 128-token prompt)	≤ 80	65
First token, fresh 128- token prompt (ms)	55	232	≤ 110	90
Prefill throughput (tokens/s)	~5,000 (FP8 compute)	not published	$\geq 3,000$	4,000
Host overhead per step (ms)	0	several (Python scheduler, eager draft head)	≤ 1	0.5
Greedy output vs plain decoding	identical	identical	identical	identical

Where the step time goes, at 4k context and a 16-row verify, target column: 60 ms for the 14.9 GB weight pass at 245 GB/s; 3–5 ms of verify compute that the pass does not hide; 4 ms for the NVFP4 drafter (1.1 GB) and its reduced-vocabulary head; 1 ms for acceptance, sampling and the one device-to-host copy of accepted tokens; under 1 ms of host work. Total about 70 ms. The public recipe spends roughly 65 ms on a 17.6 GB pass, 13 ms reading a 3.6 GB BF16 drafter, a second full LM-head pass for the draft, and several milliseconds in Python between graphs.

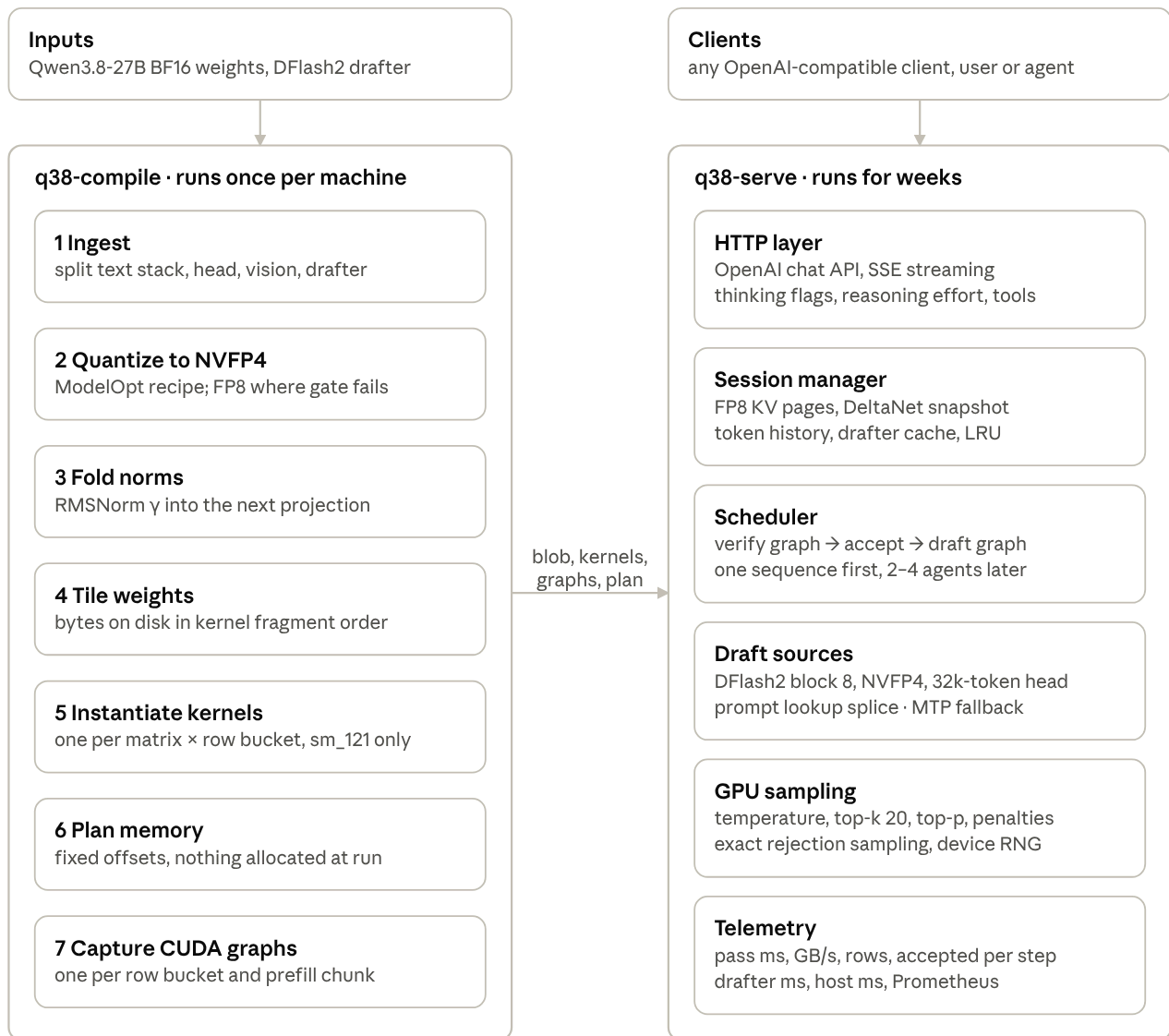
How the decode rows follow from acceptance: at 70 ms per step, 3.3 accepted tokens give 47 tokens/s, 4.3 give 61, 6.0 give 86, and 10 (a block-16 drafter or prompt lookup on an edit) give 143. The prose target therefore rests on the step-time cut alone; the code and edit targets also need the larger blocks and prompt-lookup splicing of M3.

Two things are deliberately not promised. Long-context decode slows with the KV read: at 262k tokens the pass is 23 GB and the ceiling 12 tokens/s, so all rows above scale down by about a third there. And the stretch column needs an all-NVFP4 LM head and a retrained block-16 drafter, both of which carry a quality or acceptance risk that M3 measures before anyone depends on them.

Architecture: an offline compiler and a resident runtime

Two programs. `q38-compile` runs once per checkpoint and machine and decides everything that can be decided before the first request; `q38-serve` runs for weeks and makes no decisions at all.

The compiler decides everything before the first request; the runtime only runs



compiler and runtime $\cdot$ 7 stages, 6 components

The compiler's seven stages run top to bottom once and hand the runtime a weight blob, compiled kernels, captured graphs and a memory plan; the runtime's six components then serve requests without deciding anything.

Compiler stages, in order:

1. Ingest the Hugging Face safetensors (`Qwen3_5ForConditionalGeneration`) and split them into the text stack, embedding, LM head, vision encoder, MTP module and the DFlash2 drafter.

2. Quantize to NVFP4 (E2M1 values, one E4M3 scale per 16, one FP32 scale per tensor) starting from NVIDIA's Model Optimizer mixed-precision recipe, with a calibration set that mixes code, prose, math and thinking traces. Tensors that fail the quality gate in section 10 stay at FP8. The drafter is quantized the same way; drafter error costs speed, never correctness.
3. Fold what folds: each pre-layer RMSNorm's γ is multiplied into the projection that follows it (the DeltaNet in-projection, the attention q/k/v, the FFN gate and up). The per-head q/k norms and the DeltaNet output norm act after their projections and cannot fold; they become kernel epilogues.
4. Tile: permute every matrix into the fragment order its kernel's warps will load (the SM120 block-scaled MMA layout), interleave the scales with the values, and write one 4 KB-aligned blob plus a manifest of offsets and hashes. Load is a memory map; nothing is converted.
5. Instantiate kernels: one template instance per (matrix, row bucket) with all dimensions as compile-time constants, row buckets {1, 8, 16, 32} for decode and verify and {256, 512, 1024, 2048} for prefill chunks, compiled for sm_121 only. Tile configurations are autotuned once on the Spark and baked in; there is no runtime heuristic.
6. Plan memory: fixed offsets for weights, the paged KV pool (64-token pages), DeltaNet state slots, drafter feature cache, activation scratch, logits and candidate buffers, sized from the free memory found at startup. Nothing is allocated while serving.
7. Capture CUDA graphs: one verify-plus-draft graph per row bucket and one prefill graph per chunk size. Graphs read sequence lengths and page tables from device memory, so a single graph serves every context length.

Runtime components:

- A C++/CUDA core with no Python on the hot path, behind a thin asynchronous HTTP layer that speaks the OpenAI chat-completions API with server-sent events, `reasoning_effort`, `enable_thinking`, `preserve_thinking`, and the Qwen3 tool-call format.
- A session manager keyed by conversation: paged FP8 KV, one DeltaNet snapshot slot, the conv window, the token history, and the target hidden features the drafter conditions on. Eviction is LRU inside the single memory pool; sessions sharing a system prompt share its pages.
- A scheduler with a single-sequence fast path. One step is: verify graph (which also commits the previous step's DeltaNet replay) → accept → draft graph → emit tokens. A small-batch mode for 2–4 agents is a later addition.

- Three draft sources: the DFlash2 drafter by default, a CPU n-gram matcher over the session's own tokens for prompt lookup, and the model's MTP module as a fallback if a checkpoint ships without a DFlash2 drafter.
- GPU-side sampling inside the graph: temperature, top-k 20, top-p, min-p, presence and repetition penalties (a 31 KB presence bitmap per session over the 248k vocabulary), exact rejection sampling for speculation, and device-resident RNG. One small device-to-host copy per step carries the accepted tokens.
- The Qwen 248k BPE tokenizer through the Rust `tokenizers` C bindings, with incremental detokenization for streaming.
- The vision encoder, run only at prefill when a request carries images, producing visual tokens with their M-RoPE positions.
- Telemetry per step: pass time, achieved GB/s, rows verified, tokens accepted, drafter time, host time, exported as Prometheus metrics.

Kernel plan: five kernels per layer, all shape-specialized

Each of the 64 layers runs as five fused kernels, 333 launches per step inside one CUDA graph, with no host work between them. Every kernel reads NVFP4 weights in their on-disk tile order and dequantizes in registers inside the multiply; no dequantized copy ever exists. Rows (M) are the in-flight tokens: 1 for plain decode, 8–32 for a verify.

Gated DeltaNet layer ($\times 48$):

1. `gdn_in` : RMSNorm of the input, recomputed per thread block (one 5120-value sum of squares, cheap), then the fused in-projection to q, k, v, z, β and α ($N = 16,480$). Output $M \times 16,480$ in BF16.
2. `gdn_scan` : causal conv1d of width 4 over the cached 3-token window plus the M rows, SiLU, then the gated delta rule per head over the chain from the layer's snapshot state; gated RMSNorm of the output times SiLU(z). Reads and writes one 3.1 MB FP32 state per layer; compute is trivial ($M \times 48$ heads $\times 128 \times 128$ multiply-adds).
3. `gdn_out` : out-projection ($6144 \rightarrow 5120$) with the residual add in the epilogue.
4. `ffn_up` : RMSNorm recomputed in the prologue, gate and up walked together ($N = 34,816$), SiLU(gate) $\cdot$ up in the epilogue; writes the $M \times 17,408$ activation once.
5. `ffn_down` : down-projection ($17,408 \rightarrow 5120$) with the residual add.

Gated attention layer ($\times 16$): the same FFN pair, and in place of the first three:

1. `attn_in` : RMSNorm, fused $q(+gate)/k/v$ projection ($N = 14,336$), per-head RMSNorm of q and k , partial M-RoPE on 64 of 256 dimensions, FP8 quantization of k and v and their append to the paged cache.
2. `attn` : decode attention over the paged FP8 cache plus the in-flight rows under a chain or tree mask. With 6 query heads per KV head and M rows, each KV head sees 6M query rows, enough to run on tensor cores; the FP8 values are dequantized in registers. The epilogue multiplies by $\text{sigmoid}(gate)$.
3. `attn_out` : out-projection ($6144 \rightarrow 5120$) with the residual add.

Head and speculation:

- `lm_head` : final RMSNorm plus the $5120 \times 248,320$ projection for M rows, NVFP4 or FP8 weights; logits are $M \times 248,320$ in BF16 (0.5 MB per row).
- `accept` : argmax per row and prefix comparison for greedy; for sampling, the Qwen sampling chain (temperature, presence penalty, top-k 20, top-p, min-p) applied to the target logits, then exact rejection sampling against the draft probabilities. Emits the accepted count and tokens.
- `draft_*` : the DFlash2 drafter's five sliding-window layers over the accepted tokens' target features plus 7 mask tokens, in NVFP4, then its head over a reduced vocabulary (section 8).

Two Qwen3.8-specific decisions:

- DeltaNet state and speculation. A verify runs the recurrence over M draft tokens, but only the first j are accepted, so the committed state is the one after token j . SGLang materializes every intermediate state (8×75 MB per step in BF16). We instead keep the snapshot S_0 and the M rows' k, v, α, β (130 KB per layer), and the next step's `gdn_scan` replays the j accepted tokens from S_0 before it reads the new draft, writing the result as the new snapshot. Cost: one 151 MB read and one write per step across all layers, in FP32, about 2% of the pass. Tree branches are handled the same way: each branch is a replay from S_0 , trivial compute, with S_0 served from L2.
- The 248k-token LM head is 5% of the pass at NVFP4 and 16% at BF16, and the drafter needs it a second time per step. The target head stays at NVFP4 or FP8 (quality gate decides); the drafter gets a separate head over the 32k most frequent tokens (0.16 GB in FP8), since a draft token outside that set is just a rejected guess.

What fusion buys here is modest on its own and essential in combination. A layer in a generic engine is 10–14 launches with the host encoding between them, 700–900 per step; at a few microseconds each that is several milliseconds, 5–10% of a 60 ms pass. The fused

path removes that, keeps the 17,408-wide FFN activation out of memory, and, most important, makes a 16-row verify cost 3–5 ms of visible compute instead of a second pass of memory traffic. That is what turns Spark's compute surplus into accepted tokens.

Speculative decoding: more tokens per pass

Speculation is the only multiplier on a bandwidth-bound machine, so this section is where the targets are won or lost. The design keeps DFlash2 as the drafter, because a block drafter fits this bus, and changes how it is run.

Why DFlash2 and not EAGLE-3. An autoregressive drafter makes one pass per draft token; on a 273 GB/s bus even a 1 GB drafter costs 4 ms per token, 28 ms for seven, half a target pass. DFlash2 drafts the whole block in one pass by denoising seven mask tokens conditioned on the target's hidden states, so drafting costs one small pass whatever the block size. The community measures its acceptance at ~6.0 tokens per step on math and ~3.3 on long prose, and W8 quantization of the drafter left acceptance unchanged (2.65 vs 2.72 on a prose floor).

Changes to how the drafter runs:

- Quantize it to NVFP4 (about 1.1 GB from 3.58 GiB BF16), cutting its pass from 13 ms to 4 ms. The W8 result says acceptance should survive; M3 measures the NVFP4 delta and falls back to FP8 (1.9 GB, 7 ms) if it costs more than 0.3 accepted tokens.
- Give it a reduced-vocabulary head over the 32k most frequent tokens, 0.16 GB in FP8, instead of a second pass over the 0.7–2.5 GB target head. Candidate selection runs over that subset; its top-K codebooks stay in BF16 as in the W8 recipe.
- Keep its features resident: the target hidden states it conditions on are written by the target's own kernels at the fixed layers it reads, so the drafter never recomputes anything over the context.
- Capture it in the same CUDA graph as the verify, so a step is one graph launch.

Verification shape. Row buckets of 8, 16 and 32 are compiled. The default block is DFlash2's trained 8; with its top-K candidate selector, up to 3 alternatives at the first uncertain positions form a tree of at most 16 nodes, verified under a tree mask in the attention kernels and as separate replays in the DeltaNet kernels. Because adding rows costs compute and not bandwidth, M2 measures accepted tokens per millisecond for each bucket and picks per request class; the expectation is 16 for prose and 32 for code.

Prompt lookup for edits. The Husky result that transfers directly is that edit outputs are mostly copies of the input. A CPU n-gram matcher over the session's own tokens (the last 3–4 generated tokens looked up in the context) proposes a continuation whenever it finds a match of 8 or more tokens. Those tokens are spliced after the DFlash2 block to fill the 32-row bucket, so a copy run can accept up to 31 tokens in one pass: 400+ tokens/s on a pure copy, well above the 150 tokens/s edit target. The matcher costs microseconds and runs during the previous step.

Exactness. Under greedy decoding a draft token is accepted only if it equals the target's argmax, so the output is identical to plain decoding; under sampling the accept kernel uses the standard rejection rule with the drafter's probabilities, which preserves the target distribution. The Qwen sampling chain (temperature, top-k 20, top-p, min-p, presence penalty 1.5 in instruct mode) is applied to the target logits before acceptance, so penalties are honoured exactly. The MTP module ships with the model and is kept as a fallback drafter; it is autoregressive and about one layer in size, so it drafts 3–4 tokens at modest cost and is only used if no DFlash2 drafter exists for a checkpoint.

A block-16 drafter is the main stretch item. DFlash's first release shipped block-16 drafters for Qwen3, and a retrained Qwen3.8-27B drafter at block 16 would lift code acceptance toward 10 tokens per step. Training needs the z-lab recipe and a larger GPU than the Spark for a day or two; it is scheduled in M3 and nothing before it depends on it.

Sources: [DFlash paper, arXiv 2602.06036](#) · [DFlash2 drafter W8 card](#) · [z-lab/Qwen3.8-27B-DFlash2](#)

Sessions and prefill: answer from where the chat left off

A follow-up message should cost one pass, not a reread of the conversation. Each conversation stays resident as paged FP8 KV (32 KB per token), one DeltaNet snapshot slot (151 MB), the conv window, the token history and the drafter's feature cache. A 32k-token session is about 1.2 GB, so the pool holds roughly 60 of them beside the 16 GB of weights; eviction is LRU and, because there is no separate host memory on this machine, eviction means freeing, not copying.

The follow-up path. The new message's tokens are appended to the resident state in the same pass that produces the first output token: 15 new rows and the generation row go through the model together, one weight read, 55–60 ms. With tokenization, the prefix check and sampling the target is 80 ms to the first token, against 232 ms for the public recipe on a fresh 128-token prompt.

Prefix stability. Qwen3.8 keeps earlier thinking blocks in the history by default (`preserve_thinking` on), so a conversation's token prefix does not change between turns and the cache extends cleanly. If a client turns `preserve_thinking` off, the template strips earlier thinking and the prefix changes; the runtime then re-prefills the stripped assistant turn in the background as soon as a turn completes, a few hundred tokens of compute-bound work, before the user's next message arrives. The same check guards tool results and system-prompt edits: the runtime compares the new token prefix with the stored one and re-prefills from the first divergence.

Prefill. Long prompts are compute-bound: 48.7 GFLOP per token, so 128 tokens are still one weight-bound pass (about 60 ms) and 32k tokens are about 1.6 PFLOP, 11 s at 3,000 tokens/s. Prefill runs in chunks of up to 8,192 tokens through the {256–2048}-row kernel instances with FP8 activations; DeltaNet layers use the chunked parallel form of the delta rule (64-token chunks); attention layers use a FlashAttention-style kernel that writes FP8 KV directly. The chunk size keeps activation scratch at a few hundred megabytes and lets a long prefill be interrupted by a running decode step.

Images. The vision encoder runs only at prefill for requests that carry images, in BF16, and its visual tokens take M-RoPE positions; everything downstream treats them as ordinary tokens. Video is out of scope for the first release.

Long context. The native 262,144 tokens are supported; the KV read then dominates the pass (8.6 GB), so decode slows to roughly 12 tokens/s per pass. A 4-bit KV option for sessions above 64k tokens is the mitigation, measured in M5. YaRN to 1M tokens is possible with the model card's `rope_parameters` override, costs 33 GB of KV per session, and is deferred.

Validation: exactness, quality and a benchmark that cannot flatter

Three gates, run on the Spark itself, and a release does not pass unless all three do.

Exactness. Over a fixed set of 500 prompts (code, prose, math, edits, tool calls, thinking and instruct modes), greedy output with speculation on must be token-identical to greedy output with speculation off, for every row bucket and with prompt lookup on and off. The accept kernel is unit-tested against a reference implementation of rejection sampling, and a 10,000-sample distribution test on a few short prompts checks that sampled outputs with speculation match the plain distribution within statistical noise. The DeltaNet replay is tested by comparing the committed state after a partial accept with the state from a plain run over the same tokens, bit for bit in FP32.

Quality of the quantized model. Measured against BF16 Qwen3.8-27B served from a cloud GPU on the same prompts: GPQA Diamond, LiveCodeBench v6 and IFBench subsets of 200 items each, in thinking mode at the recommended sampling settings, plus perplexity on a held-out mix. The gate is a drop of at most 1 point on each benchmark and at most 2% in perplexity; NVIDIA's own NVFP4 card reports deltas inside that range. Tensors are moved from NVFP4 to FP8 one group at a time until the gate passes, LM head first. The FP8 KV cache is checked separately at 32k and 128k context with a needle-in-a-haystack set.

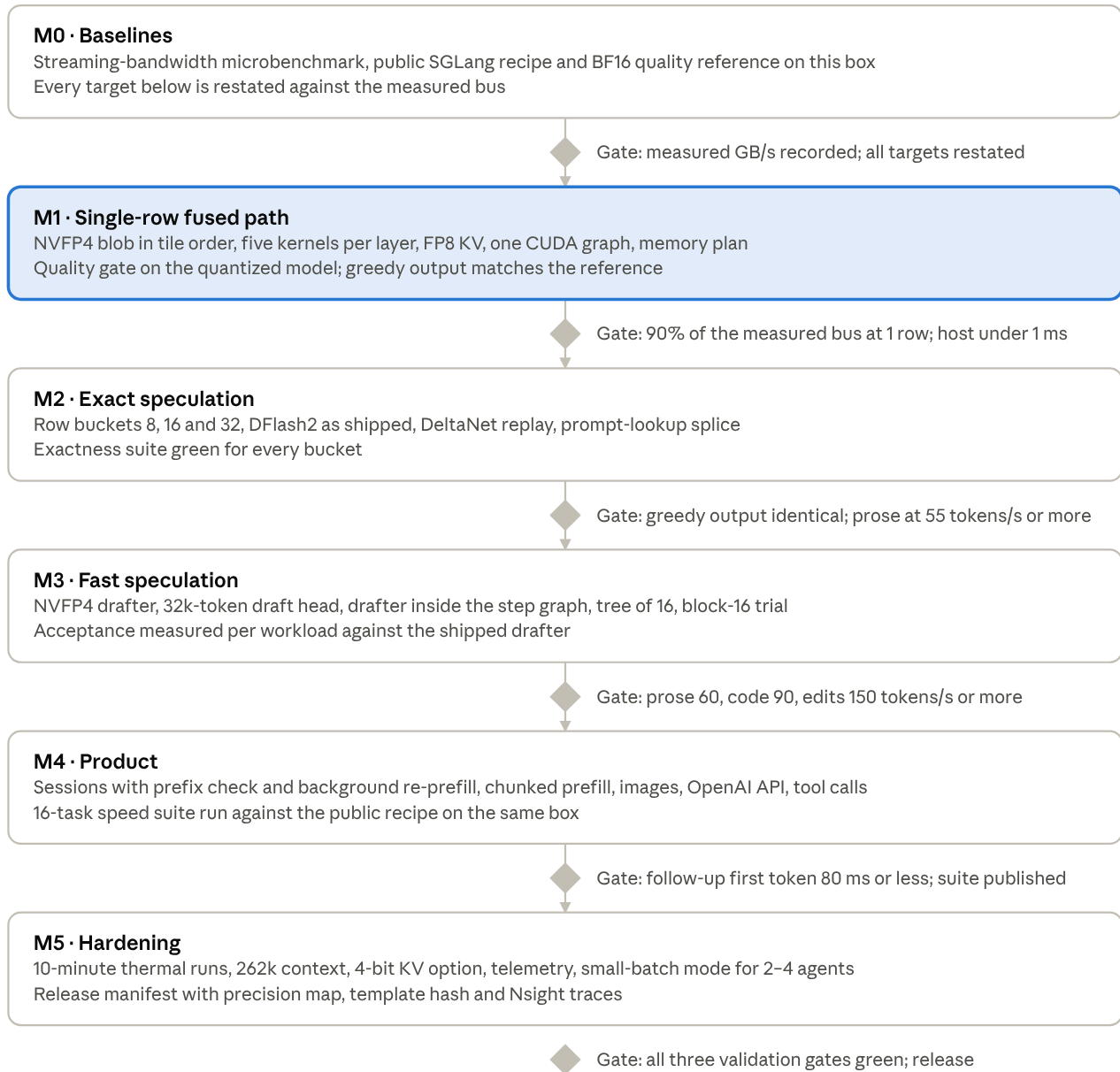
Speed. A 16-task suite modelled on Husky's, each task 20 prompts, reported as tokens/s, step time, accepted tokens per step, achieved GB/s and first-token latency: function edit, add a JSON field, rename a SQL column, write a function, fix typos, invoice to JSON, CSV to table, repeated transcript, tone rewrite, short email, project plan, meeting notes to todos, reply to an email thread, call summary, question over a 20k-token document, and a competition math problem in thinking mode. Every run records the exact prompt set and the acceptance histogram, so a headline number can always be traced to its task. The same suite runs on the public SGLang recipe on the same box for the comparison column.

Profiling method. Nsight Systems traces of a 50-step window give the per-kernel timeline and the host gaps; Nsight Compute on the five layer kernels gives `dram__bytes_read` and `dram__bytes_write`, from which achieved GB/s is computed and compared with the 245 GB/s assumption. A step is accepted as bandwidth-bound only when the weight kernels show at least 90% of the measured peak from M0's streaming microbenchmark.

- ☐ Exactness suite green for buckets 8, 16, 32
- ☐ Quality gate passed with the final tensor precision map recorded in the manifest
- ☐ Speed suite run on both engines on the same Spark, same firmware
- ☐ Nsight traces archived with the release

Milestones

Six milestones, each gated by a measurement on the Spark; M1 is the long pole, because everything after it multiplies the pass time it establishes. Rough effort for one engineer with review, labelled as an estimate: about four months end to end, M1 taking a third of it.

M1's single-row pass gates every speculation milestone

roadmap · 6 milestones, 6 gates

Each milestone ends at a measured gate; the highlighted M1 sets the pass time that M2 and M3 multiply.

M0 produces the numbers every later target is restated against: the measured streaming bandwidth, the public recipe's tokens/s and first-token latency on the same box and firmware, and the BF16 quality reference. M1 is the single-row fused path and must reach 90% of measured bandwidth before any speculation work starts, so that acceptance is

measured against a tight step and not against overhead. M2 and M3 add speculation in two stages, exact first and fast second. M4 turns the engine into a product and M5 hardens it; neither changes a kernel.

Risks and open questions

The two risks that can sink the targets are the bus and the toolchain; the rest cost time or a few percent.

Risk	Effect if it lands	Mitigation	Decided by
Sustained LPDDR5x bandwidth well below 273 GB/s (CPU traffic shares the pins)	Every tokens/s target scales down with it	Measure in M0; restate targets; keep the CPU idle during decode (no tokenization or copies on the hot path)	M0
CUTLASS and the CUDA toolchain lack mature block-scaled NVFP4 MMA kernels for sm_121	M1 slips, or decode falls back to FP8 weights (7.6 GB more per pass, ~35% slower)	Prototype the FFN kernel first, both NVFP4 and FP8, before committing the layout; FP8 is the documented fallback precision	M1
NVFP4 quality on Qwen3.8-27B misses the 1-point gate	Some tensors stay at FP8, up to +1.5 GB per pass	Start from NVIDIA's mixed recipe; move groups to FP8 LM head first; gate in section 10	M1
DFlash2 acceptance drops with an NVFP4 drafter or the 32k-token draft head	Prose tokens/s falls toward 50	Measure both changes separately; FP8 drafter and 64k head as fallbacks	M3
A block-16 drafter cannot be trained to useful acceptance	Code and edit stretch targets are not met; base targets still hold on prompt lookup	Treat as stretch; nothing before M3 depends on it	M3

Risk	Effect if it lands	Mitigation	Decided by
The DeltaNet replay interacts badly with tree speculation (per-branch replays exceed the L2 budget at 32 nodes)	Tree capped at 16 nodes	Measure state traffic in M2; the chain path is unaffected	M2
Thermal throttling at the 140 W TDP during long decodes	Pass time drifts up after minutes	Record clocks in telemetry; benchmark runs of 10 minutes, not 10 seconds	M0
The Qwen3.8 chat template or tool-call format changes in a model update	Prefix check triggers re-prefill on every turn	Pin the template hash in the manifest; re-prefill path is already required	M4

Open questions:

- ☐ Which layers' hidden states does the DFlash2 drafter condition on, exactly? Read from the z-lab config before M2; it decides which target kernels export features.
- ☐ Does the SM120 block-scaled MMA path on sm_121 reach 90% of bandwidth at M = 1, or does the 1-row bucket need a separate dequant-and-FMA kernel over the same tile layout?
- ☐ What is the first-token latency floor once tokenization and HTTP are included: is 80 ms achievable or does it settle near 90?
- ☐ Is a 4-bit KV cache needed for sessions above 64k tokens, or is FP8 enough for the users this engine serves?

Sources

Pages opened for the figures in this document, as of 8 October 2026.

- [Qwen/Qwen3.8-27B model card](#): architecture, shapes, context length, sampling and thinking settings.
- [Qwen3.8-27B on DGX Spark, 48 tok/s \(OpenZeka\)](#): the public SGLang + DFlash2 recipe, its flags, memory footprint and concurrency sweep.

- [Qwen3.8-Flash-Next on a single DGX Spark \(OpenZeka\)](#): the 176B MoE alternative at 28.5 tokens/s.
- [DFlash2 drafter, W8 card](#): drafter architecture, size, acceptance by workload, quantization result.
- [z-lab/Qwen3.8-27B-DFlash2](#) and [DFlash paper, arXiv 2602.06036](#): block-diffusion drafting.
- [NVIDIA DGX Spark hardware overview](#): memory, bandwidth, CPU.
- [NVIDIA GB10 at Hot Chips 2025](#): L2 size, coherent unified memory.
- [Ollama DGX Spark performance](#) and [llama.cpp on DGX Spark](#): what generic engines achieve on this box.
- [nvidia/Qwen3.8-27B-NVFP4](#): the mixed-precision NVFP4 recipe this design starts from.